

NDVI Geoportal, Technical Documentation

AOI clipped Sentinel-2 and Copernicus DEM on React, Node.js, GeoServer and PostGIS

Qandeel Fatima

Table of Contents

1. What this project is?	1
1.1 What the assessment asked for, and where each item lives in the app	1
2. The four pieces of software, and why we chose each one	2
2.1 PostGIS	2
2.2 GeoServer	3
2.3 Node.js / Express (the API in front of GeoServer and PostGIS)	3
2.4 React and OpenLayers (the dashboard the user sees)	3
3. What the Python pipeline does, step by step	3
3.1 Step 1: build the “area of interest” (aoi.py)	4
3.2 Step 2: pick the right Sentinel-2 photograph (stac_search.py)	4
3.3 Step 3: download the raw satellite bands (download.py)	4
3.4 Step 4: build the elevation map (dem.py)	5
3.5 Step 5: build the RGB and NDVI images (process.py)	5
3.6 Step 6: put everything into the database (load_postgis.py)	6
3.7 Step 7: publish everything in GeoServer (setup_geoserver.py)	7
4. The database in detail	7
4.1 sample_points	7
4.2 aoi_boundary	7
4.3 contours	7
4.4 app_meta	7
5. The API endpoints in detail	8
5.1 What the identify endpoint does, step by step	8
5.2 Why this is safe	9
6. The dashboard in detail	9
6.1 What each chart shows	9
6.2 The map card	9
6.3 The inspection log	10
7. Motion design (how the dashboard animates)	10
7.1 What went wrong in the first version, and how it was fixed	10
8. Performance notes	11
9. Testing	11

10. How to verify the running system	12
11. UI and UX notes	12
12. What we could not do inside the timeboxed session	13
13. Where each piece of code lives	13

1. What this project is?

The NDVI Geoportal is a website that shows a satellite view of Islamabad and Rawalpindi together with three “extra layers of information” that you can turn on and off:

1. A colour photograph taken from space (Sentinel-2, from the European Space Agency).
2. A “greenness map” called NDVI, which uses the same satellite image to show how much healthy plant life is on the ground.
3. An elevation map (a Digital Elevation Model, or DEM), which shows how high the ground is, plus a “hillshade” that adds soft shadows to make the terrain look 3D, and contour lines like on a hiking map.

On top of all that we also show about 1.9 million tiny dots. Each dot is one sample the pipeline took from a specific pixel of the satellite image, and each dot carries values that describe that spot: greenness, colour, elevation, and a “quality flag” that tells you whether the pixel was clear sky, water, cloud, and so on.

You can click anywhere on the map and the page will tell you what is under your click: the nearest sample point (with its full attribute table), the greenness value at that spot, the raw colour values, and the elevation.

1.1 What the assessment asked for, and where each item lives in the app

Assessment requirement	How this project meets it
Plot a raster image and a vector layer on a map.	The satellite image is a raster. The 1.9 million sample dots are the vector layer. Both are drawn on an OpenLayers map.
Raster and vector each larger than 200 MB.	The Sentinel-2 image is 545 MB on disk. The database table that holds the dots is 309 MB. Both numbers are on the dashboard.
Vector has at least 50,000 features.	The vector layer has 1,910,911 dots.

Assessment requirement	How this project meets it
Show pixel or feature values when the user clicks.	Clicking the map calls the API. The API returns the nearest dot, the greenness at that point, the colour values, and the elevation. All four appear in the “Inspection log” panel.
User can toggle layers on and off.	The floating panel over the map has a switch and an opacity slider per layer. The Sentinel-2 layer also has a “true colour, false colour” toggle.
Basemap switcher.	The panel at the top of the map card offers Dark, Imagery, Light, and OSM.
Built with React, Node.js, GeoServer, PostGIS.	All four are used; each is described in section 2.
Code quality.	Every module has comments that explain the “why”, ESLint runs clean, and there are unit tests both in the API and a synthetic-data test for the Python pipeline.

2. The four pieces of software, and why we chose each one

The application has four running services. Each does one job. Splitting them keeps the code understandable and each service can be replaced later without rewriting the others.

2.1 PostGIS

What it is. PostgreSQL is one of the most widely used open-source databases in the world. PostGIS is an add-on that teaches PostgreSQL about coordinates, distances, and shapes. Together they can store 1.9 million dots and answer a question like “what is the nearest dot to this location?” in a few milliseconds.

Why we use it here. Most databases can store latitude and longitude in two columns, but they cannot answer “nearest neighbour” fast on millions of rows. PostGIS builds a special spatial index (called GiST) that acts like a folder-within-folder tree over 2D space, so lookups skip the parts of the map that are far from the click.

Where it lives in the repo. It runs inside Docker as the `postgis` container. The database is called `gis`, the user is `gis`, and the password is set in the `.env` file.

2.2 GeoServer

What it is. GeoServer is a Java server that reads spatial files (like the Sentinel-2 image on disk) or a database table, and produces map images on demand. When the browser shows a tile of the map, it is actually asking GeoServer, “please render this small square of the world as a PNG”.

Why we use it here. The browser cannot render a 500 MB image directly. GeoServer takes only the pixels the user is looking at, applies a colour rule (a “style”), and returns a small tile (usually 256 by 256 pixels). This makes the app feel fast even when the underlying files are huge.

Styles. Every layer has a “style” written in a language called SLD (Styled Layer Descriptor). The style says things like “colour NDVI values from blue at -0.2 to dark green at 0.9”. All styles live in `prep/styles/*.sld` and are uploaded to GeoServer by the Python pipeline.

2.3 Node.js / Express (the API in front of GeoServer and PostGIS)

What it is. Node.js is a runtime that lets us write server code in JavaScript. Express is a small library that helps us define URLs like `/api/health` and match them to functions.

Why we use it here. The browser must never talk to GeoServer or PostGIS directly, because that would open the door to running arbitrary queries from anywhere on the internet. Instead, the browser talks only to Node, and Node decides what to allow. Node also caches heavy queries in memory so we do not re-run them on every page load.

2.4 React and OpenLayers (the dashboard the user sees)

What it is. React is a library for building user interfaces out of small reusable pieces called components. OpenLayers is a library that specialises in maps: it knows how to fetch tiles, pan, zoom, and turn a click into a coordinate.

Why we use it here. The dashboard has many small parts (the header, four charts, the map, the log). React makes it easy to change one part without affecting the others. OpenLayers is the industry standard for open-source web mapping and it has excellent support for the exact WMS format GeoServer speaks.

3. What the Python pipeline does, step by step

The pipeline is a Python program that runs once in its own Docker container. Its job is to turn “an idea” (I want a map of Islamabad and Rawalpindi) into “files and database rows that GeoServer can serve”.

It runs seven steps in order. If a step fails, you can re-run only the remaining steps with `--skip`. This section explains each step in words a non-programmer can follow, then names the file it lives in.

3.1 Step 1: build the “area of interest” (aoi.py)

An “area of interest” (AOI) is a polygon that says “we only care about the ground inside this shape”. For us, that shape is Islamabad Capital Territory plus Rawalpindi, joined into one bigger shape.

The pipeline gets this shape in one of two ways:

- If you drop a .geojson, .gpkg, or .shp file into data/aoi/, the pipeline uses that.
- Otherwise it downloads a public dataset called geoBoundaries (licence CC-BY 4.0). This dataset lists every administrative boundary in every country. We pick the two features whose names are exactly “Islamabad Capital Territory” and “Rawalpindi”, then merge them into one shape.

There is a small twist. The geoBoundaries file lives on GitHub, but the usual URL returns a tiny text pointer file instead of the real data (a Git feature called LFS). We work around this by using a different GitHub URL that returns the real data.

The final shape is saved to data/aoi/aoi.geojson. Its area is about 6,853 square kilometres, roughly the size of a small country like Brunei.

3.2 Step 2: pick the right Sentinel-2 photograph (stac_search.py)

Sentinel-2 is a pair of satellites (2A and 2B) run by the European Space Agency. They photograph the whole planet every few days. The pictures are free and there is an index of them called STAC (SpatioTemporal Asset Catalog) that anyone can search.

There is a problem though: the satellite does not take one big picture of Pakistan. It splits the ground into map squares (called MGRS tiles), and each pass photographs a strip of tiles. Our AOI is bigger than any single tile, so we need several tiles from the same day and then we glue them together (see step 5).

The pipeline searches STAC for tiles that intersect our AOI in a chosen date range (October 2025 to March 2026 by default, because winter has fewer clouds). It groups the tiles by acquisition date, and for each date checks whether the tiles together fully cover the AOI. It then picks the date with the fewest clouds across its tiles.

In this run the pipeline chose 2025-10-13, four tiles: 43SBS, 43SBT, 43SCS, 43SCT, all with 0.00% cloud cover.

3.3 Step 3: download the raw satellite bands (download.py)

For each selected tile the pipeline downloads five files:

- B02, B03, B04: the blue, green, and red colour bands.
- B08: the near-infrared band. Plants reflect a lot of near-infrared, so this band is what makes NDVI possible.
- SCL: a “scene classification” band, where every pixel is a label like “cloud” or “vegetation” or “water”.

Each file is 260 to 280 MB. Total download is about 1.3 GB.

Downloads write to a temporary .part filename first, and only get renamed to their final name when they finish. That way, if the internet drops halfway, you re-run the pipeline and it continues where it stopped.

3.4 Step 4: build the elevation map (dem.py)

The satellite photograph does not tell you elevation. For that we use the Copernicus Digital Elevation Model (Cop DEM GLO-30), also free. It is stored as one file per one-degree square. Our AOI touches three of these squares, so we download three files.

The pipeline then:

1. Stitches the three files together into one (a “mosaic”).
2. Reprojects them to a common coordinate system (EPSG:32643, called UTM zone 43N). This is a projection that stretches the world so that distances in metres are correct near Pakistan.
3. Clips the result to the AOI shape (throws away pixels outside).
4. Writes the final file as a Cloud Optimized GeoTIFF (a version of GeoTIFF that lets programs read a small region without downloading the whole file).

It also produces two extras from the same data:

- A “hillshade”, a black-and-white image that adds soft shadows as if the sun were shining from the northwest. This makes the terrain look 3D.
- Contour lines every 100 metres. Every 500 metres the line is drawn thicker and labelled, exactly like a hiking map.

Finally, for the dashboard, the pipeline downsamples the elevation grid into a small 42-row by 140-column matrix. This tiny matrix is what powers the “hero” chart on the left of the dashboard (the ridgeline / joy plot).

3.5 Step 5: build the RGB and NDVI images (process.py)

This is the biggest step. The pipeline takes the four Sentinel-2 bands (B02, B03, B04, B08) from each of the four tiles and builds two output files that cover only the AOI:

- `rgb_s2.tif`: a true-colour image, four bands (red, green, blue, near-infrared). Bands 1 to 3 are the true-colour photograph; band 4 (near-infrared) is kept so we can offer a “false-colour” style later.
- `ndvi_s2.tif`: the greenness map, one band, values from -1 (water) to +1 (dense vegetation).

The way the pipeline does this is important, so here is a plain-language walkthrough:

1. It figures out the exact grid the final image should sit on. It uses the AOI’s bounding box in UTM 43N, rounded outward to the nearest 20 metres. Rounding to 20 m matters because the SCL band’s pixels are 20 m wide. Aligning to 20 m means no half-pixel offsets and no need to resample.

2. It divides the output image into 1024 by 1024 pixel windows, and processes one window at a time. This keeps memory usage low, so even a laptop can run it.
3. For each window, it reads the same rectangle from every tile, using zero for “outside the tile”. Then it combines them with a rule “first non-zero pixel wins”. This glues the four tiles into a seamless mosaic.
4. It applies a small correction that Sentinel-2 requires: for scenes processed after January 2022 the raw values include a fixed +1000 offset that has to be subtracted.
5. It computes NDVI from the red and near-infrared bands using the classic formula $NDVI = (NIR - Red) / (NIR + Red)$.
6. It hides bad pixels (clouds, shadows, saturated pixels) by looking at the SCL band and setting NDVI to a “no data” flag for those pixels.
7. Any pixel outside the AOI becomes “no data” for every layer.
8. It then randomly picks one pixel out of every six by six block and records its values. This is what gives us the 1.9 million sample dots.

Once the raw output files are written, the pipeline runs one more small trick called adaptive compression. The rule is “the RGB file must end up at least 200 MB”. Modern compression is very good, and on some scenes the DEFLATE codec squishes the file below 200 MB. If that happens, the pipeline retries with a weaker codec (LZW), and then a weakest one (no compression at all), keeping the first codec whose output meets the size floor. This scene came out at 545 MB with DEFLATE, so no fallback was needed.

3.6 Step 6: put everything into the database (load_postgis.py)

This step opens a connection to PostGIS and does five things:

1. Runs a “schema” SQL script that says “make sure the tables exist with these columns”. This script is idempotent, which means you can run it again and again without breaking anything.
2. Loads the 1.9 million dots into the sample_points table using PostgreSQL’s fast bulk-loader called COPY.
3. Builds a spatial index (GiST) on the geometry column so nearest-neighbour queries stay fast.
4. Loads the AOI polygon into a table called aoi_boundary, and the 23 contour lines into a table called contours.
5. Writes a small “summary” into a key-value table called app_meta. The dashboard reads this summary in one shot, so the header and counters do not need to scan the giant sample_points table.

3.7 Step 7: publish everything in GeoServer (setup_geoserver.py)

The final step tells GeoServer where every file is and how to render it. It talks to GeoServer over HTTP (GeoServer has a REST API). For each of the seven layers it:

1. Uploads the style file (SLD).
2. Creates a “store” (a pointer to a file or a database).
3. Publishes the layer, wired to the store.

4. Attaches the style as the default for that layer.

For the RGB layer it also registers a second style called `false_color`. This lets the web app request a false-colour render (vegetation shows up as bright red) without any extra server work.

4. The database in detail

There are four tables in the gis database.

4.1 sample_points

Every row is one dot. There are 1,910,911 rows. The interesting columns are:

- `lon`, `lat`, `geom`: the dot's location. `geom` is derived from `lon` and `lat` automatically.
- `red`, `green`, `blue`, `nir`: the raw reflectance values at that pixel, scaled by 10,000 (so a value of 1200 means 0.12).
- `ndvi`: the greenness value, between -1 and +1.
- `ndvi_class`: a short label ("water", "bare_built", "sparse", "moderate", "dense") based on the value.
- `scl`, `scl_label`: the quality flag for that pixel.
- `pixel_row`, `pixel_col`: which pixel of the source raster it came from.
- `acquired`, `tile`: the date it was captured and which MGRS tiles were mosaicked.
- `elevation`: metres above sea level, from the DEM.

4.2 aoi_boundary

One row that holds the polygon shape of the AOI, plus its name, its source (geoBoundaries), and its area in square kilometres.

4.3 contours

One row per contour line. Each row has an elevation value in metres, a flag saying whether it is a "major" contour (every 500 m), and the line shape.

4.4 app_meta

A key-value store. Every value is a JSON blob. The dashboard reads keys like `scene`, `points`, `rasters`, `aoi`, and `terrain` to build the header card and the ridgeline chart in one HTTP request, without needing to scan the giant points table.

5. The API endpoints in detail

Every URL below returns JSON. All URLs starting with `/api` are meant for the web dashboard. The URL `/geoserver/*` is a safe proxy that lets the dashboard fetch map tiles from GeoServer without exposing GeoServer directly.

HTTP Method	Endpoint Path	Cached	Purpose
GET ▾	/api/health	No	Is the server up, is the database reachable.
GET ▾	/api/layers	10 min	List of layers, WMS URL, and the data extent.
GET ▾	/api/stats	10 min	Total dots and counts per NDVI class.
GET ▾	/api/overview	10 min	Everything the header needs in one call.
GET ▾	/api/aoi	10 min	The AOI polygon as GeoJSON.
GET ▾	/api/charts	10 min	Histogram, NDVI-by-elevation, class share.
GET ▾	/api/identify?lon=&lat=&res=&layers=	No	Four lookups combined into one answer.

5.1 What the identify endpoint does, step by step

1. Read lon, lat, res, and the list of layers from the URL. Reject anything that is not a real number in the right range.
2. Compute a search radius. If the user is very zoomed in, look only 20 m around the click. If they are very zoomed out, look up to 1 km around it.
3. In parallel, run four lookups:
 - o Ask PostGIS for the nearest dot inside the radius (uses the GiST index).
 - o Ask GeoServer for the NDVI pixel value at that spot.
 - o Ask GeoServer for the RGB and near-infrared pixel values.
 - o Ask GeoServer for the elevation pixel value.
4. Combine the four answers into one JSON reply. If one of the four fails, the reply still returns the other three, plus an error field on the failing one.

5.2 Why this is safe

Every value the user sends is validated first. Every SQL query uses placeholders (like \$1, \$2) instead of string concatenation, which rules out SQL injection. The proxy in front of GeoServer only forwards read-only WMS requests to our own workspace. Any other request gets a 403 reply.

6. The dashboard in detail

The dashboard is one page laid out as follows:

- Top bar with a pill navigation and a “Fit AOI” button.
- Header card with the AOI name, the scene date, the area, the elevation range, and the tech stack chips.
- Two columns:
 - Left: four charts (Terrain ridgeline, NDVI distribution, NDVI by elevation, land-cover share).
 - Right: the map card, then the inspection log.

6.1 What each chart shows

- **Terrain ridgeline.** A “joy plot” of the DEM. Each row is one horizontal strip of the AOI. The higher the ground on that strip, the higher the line rises above its baseline. It is the “hero” visual because it is the only chart that shows the whole shape of the landscape at a glance.
- **NDVI distribution.** A vertical histogram, 24 bars, each covering 0.05 units of NDVI from -0.2 to 1.0. The tallest bar is highlighted in lime so you can see the “most common” greenness value.
- **NDVI by elevation.** One row per 100-metre elevation band. Each row shows a capsule from the 10th to the 90th percentile of NDVI for that band, and a lime dot at the mean. Rising elevation and falling greenness (or the opposite) is easy to spot.
- **Land-cover share.** Each row is one NDVI class (water, bare, sparse, moderate, dense). The bar length is the percentage. Numbers are shown on the right, tabular-aligned.

6.2 The map card

The map card has:

- A “segmented control” for the basemap: Dark (default), Imagery, Light, OSM.
- A floating “glass” panel over the map. This panel has a toggle switch and an opacity slider for every layer. It also has a “True / False” segmented control that flips the RGB layer between true-colour and false-colour rendering.
- A coordinates readout at the bottom that shows the exact lon and lat under the mouse.
- A small NDVI legend chip in the bottom-right.

6.3 The inspection log

Every time you click on the map, one entry is added to the top of the log. The entry shows the time, the coordinates, the NDVI value with its colour swatch, the class name, the elevation, the reflectance values, the distance to the nearest sample point, and the acquisition date. The list keeps the last 10 clicks. A “Clear” button empties it.

7. Motion design (how the dashboard animates)

The dashboard is styled to feel calm and premium. Motion helps that feeling but should never get in the way. We use a small library called GSAP (short for GreenSock Animation Platform).

Concrete animations:

- On first render, the top bar and every card fades up from below with a 0.6 second animation, staggered so each card starts 0.08 seconds after the previous one.
- The area and elevation numbers in the header animate from 0 to their final value (a “counter” effect) over 1.1 seconds.
- The histogram bars grow up from the baseline (0.6 s, staggered).
- The NDVI-by-elevation capsules grow in width from the left, and the mean dots pop in slightly after with a soft “back” easing.
- The ridgeline paths draw themselves from back to front (using the stroke-dashoffset trick, which works reliably on SVG).

If the user has “reduce motion” turned on in their operating system (a real accessibility setting on Windows, macOS, iOS, and Android), the code detects it via `prefers-reduced-motion` and skips every animation. Everything is rendered in its final state immediately.

7.1 What went wrong in the first version, and how it was fixed

An earlier version of the dashboard used a common trick: it set `transform: scaleY(0)` on each SVG rectangle as the “hidden” state, then animated it to `scaleY(1)`. On a normal HTML div this works fine, because the transform is applied relative to the div’s own box.

SVG is different. On SVG elements, CSS transforms are applied relative to the outermost SVG viewport by default, not to the element’s own bounding box. That means “scale from the bottom” actually collapses the shape off-canvas, and the browser never draws anything. The fix is one CSS property: `transform-box: fill-box`, which tells the browser to use the element’s own bounding box instead. Now the animation lands where you expect.

Along the same fix, the charts were rewritten to use React’s `useGSAP` hook (from `@gsap/react`). This hook is aware of React’s lifecycle: it disposes tweens on unmount, and it re-runs the animation when the data changes. That way, when the API data arrives after the first render, the entrance animation replays and the charts are always visible.

8. Performance notes

- **Windowed processing.** The pipeline never loads a full raster into memory. It processes 1024 by 1024 pixel windows at a time, so memory usage stays flat at a few dozen megabytes even for a full mosaic.
- **Grid alignment.** Because the output grid is aligned to 20 metres, the source pixels line up exactly and no resampling happens. Nearest-neighbour reads only.

- **Retryable downloads.** Each downloaded file writes to a .part filename first and only gets its final name once complete. The pipeline also retries a failed download with exponential backoff. Two network drops during this run were handled automatically.
- **CLUSTER on the spatial index.** After the bulk load, PostGIS is asked to physically reorder rows so that dots that are close on the map are also close on disk. WMS tiles then touch fewer pages when GeoServer reads the table.
- **In-memory cache.** The API caches the “overview” and “charts” responses for 10 minutes. The pipeline only writes the database once, so this cache is safe.
- **Parallel identify.** The identify endpoint runs its four lookups (points, NDVI, RGB, DEM) at the same time, not one after the other. One slow layer never blocks the others.
- **Tiled WMS.** The map uses 256 by 256 tiles. Browsers cache them and GeoServer’s own tile cache (GeoWebCache) can serve repeats without re-rendering.

9. Testing

There are three test suites.

API unit tests (cd api && npm test). Covers input validation, GeoServer URL building, the proxy whitelist, the histogram helper, and the shape of every payload (checking that pg’s bigint/numeric columns come out as JavaScript numbers, not strings). 15 tests, all pass.

API lint (cd api && npm run lint). Runs ESLint on the API. Zero errors.

Synthetic pipeline test (docker compose run --rm --entrypoint python prep test_synthetic.py). Builds two fake Sentinel-2 tiles in code, a tiny fake DEM, and an AOI that crosses the boundary between them. Runs the process step and asserts:

- Both tiles contributed to the mosaic (each tile’s distinctive value is present).
- Pixels outside the AOI are set to no-data.
- Points were sampled only inside the AOI.
- Every sampled point carries a finite elevation.
- The RGB file has 4 bands.

This test takes about 10 seconds and runs without touching the internet.

Web build and lint (cd web && npm run lint && npm run build). ESLint passes. Vite builds the app to dist/ (about 585 KB, or 183 KB gzipped).

10. How to verify the running system

After the pipeline has finished, this is the “smoke test” I ran:

```
docker compose up -d --build api web
```

```
curl -s http://localhost:4000/api/health
curl -s http://localhost:4000/api/overview | jq .
curl -s http://localhost:4000/api/charts | jq '.ndviClassShare'
```

```
curl -s http://localhost:4000/api/aoi | jq '.properties'
curl -s 'http://localhost:4000/api/identify?lon=73.05&lat=33.70&res=9.5' | jq .
```

```
docker compose exec postgis psql -U gis -d gis -c "
SELECT count(*),
       pg_size_pretty(pg_relation_size('sample_points')),
       pg_size_pretty(pg_total_relation_size('sample_points'))
FROM sample_points;"
```

```
ls -lh data/rasters
cat data/manifest.json
```

Results from the current build:

- /api/health: {"status":"ok","database":"up"}
- /api/overview: AOI Islamabad + Rawalpindi, 6852.8 km2, scene 2025-10-13, 4 tiles, 1,910,911 points, RGB 545 MB, DEM 25 MB, elevation 319 to 2670 m.
- /api/identify at (73.05, 33.70): nearest point 18 m away, class “moderate”, NDVI 0.599, elevation 551.5 m.
- Database: 1,910,911 rows, 309 MB table (480 MB with indexes).
- Files on disk: rgb_s2.tif 520 MB, ndvi_s2.tif 279 MB, dem_cop30.tif 25 MB, hillshade.tif 6.4 MB.

Every assessment size requirement is met.

11. UI and UX notes

- The single source of truth for colours is web/src/utils/palette.js. The SLDs use the same colour stops. That way a bar in a chart uses exactly the same colour as the same pixel on the map.
- Every number on the dashboard uses tabular figures. This is a font feature (font-feature-settings: 'tnum') that makes digits equal-width. When the counter animates from 0, digits do not shift around; they update in place.
- The whole dashboard collapses to a single column below 900 px width. The map card stays at least 420 px tall so it remains usable on a small laptop screen.

12. What we could not do inside the timeboxed session

- The pipeline needed a stable network for around one hour of downloads. Two brief network drops were survived automatically by the retry logic; a longer outage would still stop the pipeline.
- The dashboard was verified using curl (the API endpoints) and by inspecting the built JavaScript bundle. It was not driven end-to-end in a headless browser inside the session. If you want automated UI tests, add Playwright to the web workspace.
- The web bundle is 585 KB. That is above Vite’s default warning threshold of 500 KB, mainly because OpenLayers is a big library. For a production release, splitting OpenLayers and GSAP into their own chunks with

`build.rollupOptions.output.manualChunks` would drop the initial download by roughly a third.

13. Where each piece of code lives

File/Path	Description
<code>ndvi-geoportal/</code>	Root directory
<code>docker-compose.yml</code>	four services in one file
<code>.env.example</code>	copy to <code>.env</code> , edit passwords / ports if you like
<code>db/init/01_schema.sql</code>	DB schema that runs once on a fresh volume
<code>prep/</code>	the Python pipeline
<code>run_all.py</code>	chains the 7 steps
<code>config.py</code>	every tunable value in one place
<code>aoi.py</code>	step 1 (AOI)
<code>stac_search.py</code>	step 2 (pick scene)
<code>download.py</code>	step 3 (download bands, retry on network drops)
<code>dem.py</code>	step 4 (DEM, hillshade, contours, ridgeline)
<code>process.py</code>	step 5 (mosaic + RGB + NDVI + point sample)
<code>load_postgis.py</code>	step 6 (bulk load + AOI + contours + meta)
<code>setup_geoserver.py</code>	step 7 (publish layers + styles)
<code>schema.sql</code>	idempotent DB schema (applied every run)
<code>test_synthetic.py</code>	offline pipeline smoke test
<code>styles/*.sld</code>	one file per GeoServer style
<code>api/</code>	the Node.js API

File/Path	Description
src/app.js	Express middleware wiring
src/routes/	identify + meta (health, layers, stats, overview, aoi, charts)
src/services/	PostGIS queries + GeoServer callers + charts
src/middleware/	proxy allowlist, error handler
test/	unit tests
web/	the React app
src/App.jsx	dashboard shell
src/components/	TopBar, HeaderComponent, LayerPanel, MapCard, InspectionLog
src/components/charts/	hand-built SVG charts
src/map/	OpenLayers wiring + layer factories
src/utils/	palette, format, motion helpers
docs/	DOCUMENTATION.md (this file) + docx export